

AI Extension Update for Playwright TypeScript Implementation:

Ask:

Read my entire workspace to understand and analyse how java selenium is implemented in the code base.

Based on above analysis do let me know what are all impacted files.

My objective is that I want to implement Playwright Typescript same as Selenium Java implementation done here.

Agent:

Prompt.js change: Go to Prompt.js and add the missing templates for Playwright TypeScript

Chat.js change:

Go to chat.js and update below changes required for Playwright TypeScript implementation:

1. Add isPlaywrightTypeScript() method (similar to isJavaSelenium())
2. Update getPromptKeys() to route TypeScript/Playwright combinations to the new prompts
3. Remove the "unsupported" warning for TypeScript/Playwright combinations

Implementation Summary

Your Playwright TypeScript implementation is now **complete!** The system now supports:

Language	Engine	Feature File	Page Object	Combined
Java	Selenium	✓	✓	✓
TypeScript	Playwright	✓	✓	✓

How It Works:

When a user selects **TypeScript + Playwright**:

- **Feature Only** → CUCUMBER_ONLY prompt
- **Page Only** → PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY prompt (new)
- **Both** → CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS prompt (new)

All new prompts with proper Playwright 1.40+ syntax, TypeScript types, and Cucumber integration are ready to generate code! 🚀



Code Generator

Settings

Supported Language

TypeScript



Browser Engine

Playwright



Select LLM Provider

Groq



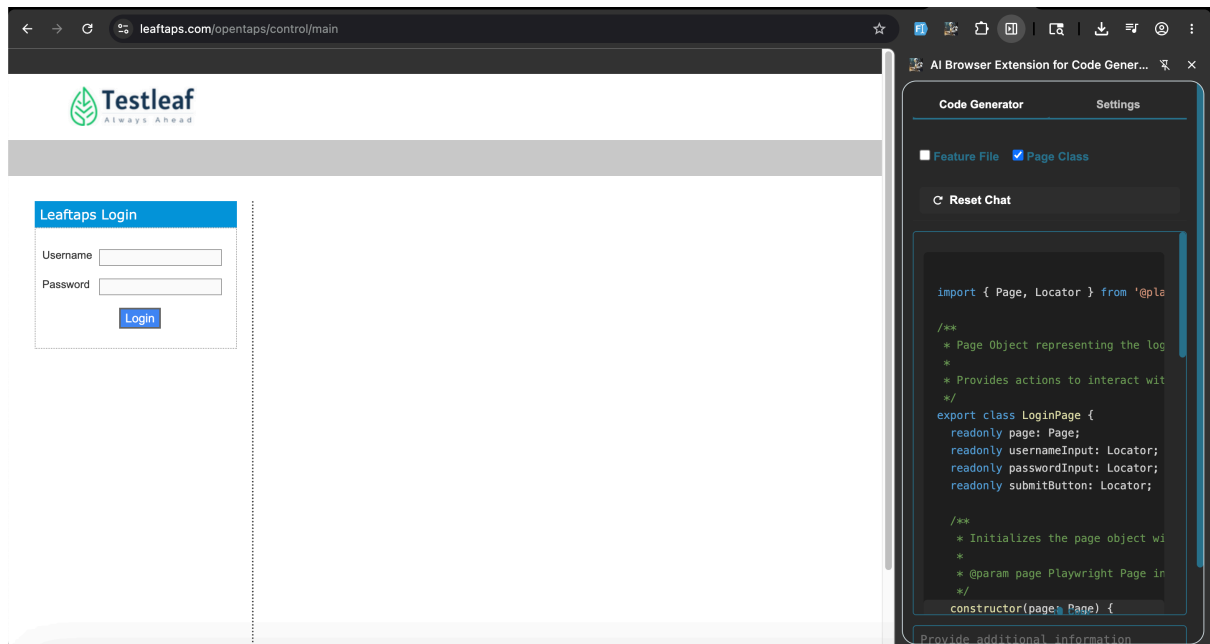
Select Model

OpenAI GPT-OSS-120B



Groq API Key

.....



Page Class:

```
import { Page, Locator } from '@playwright/test';

/**
 * Page Object representing the login form.
 *
 * Provides actions to interact with username, password fields and submit the form.
 */
export class LoginPage {
  readonly page: Page;
  readonly usernameInput: Locator;
  readonly passwordInput: Locator;
  readonly submitButton: Locator;

  /**
   * Initializes the page object with the given Playwright page.
   *
   * @param page Playwright Page instance.
   */
  constructor(page: Page) {
    this.page = page;
    this.usernameInput = page.locator('input#username');
    this.passwordInput = page.locator('input#password');
    this.submitButton = page.locator('input.decorativeSubmit[type="submit"]');
  }

  /**
   * Types the provided username into the username field.
   *
   * @param username The username to enter.
   */
  async setUsername(username: string): Promise<void> {
    await this.usernameInput.fill(username);
  }
}
```

```

/**
 * Types the provided password into the password field.
 *
 * @param password The password to enter.
 */
async setPassword(password: string): Promise<void> {
  await this.passwordInput.fill(password);
}

/**
 * Clicks the login (submit) button to submit the form.
 */
async submitLogin(): Promise<void> {
  await this.submitButton.click();
}

/**
 * Performs a full login flow using the supplied credentials.
 *
 * @param username The username to use.
 * @param password The password to use.
 */
async loginAs(username: string, password: string): Promise<void> {
  await this.setUsername(username);
  await this.setPassword(password);
  await this.submitLogin();
}
}

```

Feature File:

The screenshot shows the Testleaf web application interface. The main content area displays a login form titled "LeafTaps Login" with fields for "Username" and "Password", and a "Login" button. The sidebar on the right contains a "Code Generator" extension with a "Settings" tab. The "Feature File" tab is selected, showing a Gherkin feature file for "Login to OpenTaps".

Feature: Login to OpenTaps

Scenario Outline: Successful login
 Given I open the login page
 When I type "<username>" into the username field
 And I type "<password>" into the password field
 And I click the Login button
 Then I should be logged in successfully

Examples:

username	password
"raskumar"	"Rk@202"
"sri.priya"	"Priya@"
"kavitha_raj"	"Kavitha@"
"arun.murthy"	"Arun!4"

Scenario Outline: Login attempt
 Given I open the login page
 When I type "<username>" into the username field

Feature: Login to OpenTaps

Scenario Outline: Successful login with valid South Indian credentials
 Given I open the login page

When I type "<username>" into the Username field
And I type "<password>" into the Password field
And I click the Login button
Then I should be logged in successfully

Examples:

username	password	
"ramkumar"	"Rk@2023!"	
"sri.priya"	"Priya#123"	
"kavitha_raj"	"Kavitha\$99"	
"arun.murthy"	"Arun!456"	

Scenario Outline: Login attempt with incorrect password

Given I open the login page
When I type "<username>" into the Username field
And I type "<wrong_password>" into the Password field
And I click the Login button
Then I should see an authentication error message

Examples:

username	wrong_password	
"ramkumar"	"wrong123"	
"sri.priya"	"12345"	
"kavitha_raj"	"password"	
"arun.murthy"	"abc!@#"	

Scenario Outline: Login attempt with missing username

Given I open the login page
When I leave the Username field empty
And I type "<password>" into the Password field
And I click the Login button
Then I should see a validation message for the Username field

Examples:

password	
"Rk@2023!"	
"Priya#123"	
"Kavitha\$99"	
"Arun!456"	

Scenario Outline: Login attempt with missing password

Given I open the login page
When I type "<username>" into the Username field
And I leave the Password field empty
And I click the Login button
Then I should see a validation message for the Password field

Examples:

username	
"ramkumar"	
"sri.priya"	
"kavitha_raj"	
"arun.murthy"	

Test Script Generator:

Read my entire workspace to understand and analyze how feature file and page class is implemented in the code base

Based on above analysis do let me know what are all impacted files.

My objective is that I want to implement Test Script code generation using Playwright Typescript

Comprehensive Analysis: Feature File & Page Class Implementation

I've analyzed the entire workspace to understand how feature files and page classes are currently implemented. Here's the complete breakdown:

Current Implementation Architecture

1. Generation Modes (UI Controls)

Located in [panel.html](#) lines 33-47:

- ☒ Feature File (checked by default)
- ☐ Page Class

Users can check any combination of these two options to generate different outputs.

2. Code Generation Request Flow

Panel.html (User selects)

↓

[Language: Java/C#/Python/TypeScript]

[Engine: Selenium/Playwright/Cypress/Puppeteer]

[Modes: ✓Feature, ✓Page, etc.]

↓

chat.js - sendMessage() (Captures DOM + user instructions)

↓

getPromptKeys(language, engine) - Routes to correct templates

↓

prompts.js - getPrompt() - Fills template variables (\${domContent}, \${pageUrl})

↓

API Call (Groq/OpenAI/Testleaf)

↓

Response displayed with syntax highlighting + copy button

. How Mode Selection Works

[chat.js](#) lines 551-602 - [getPromptKeys\(\)](#) method:

```
// Reads checked boxes from panel.html
const checkboxes = Array.from(
  document.querySelectorAll('input[name="javaGenerationMode"]:checked')
);
const isFeatureChecked = checkboxes.some(box => box.value === 'FEATURE');
const isPageChecked = checkboxes.some(box => box.value === 'PAGE');

// Returns appropriate template keys based on selection
if (isFeatureChecked && isPageChecked) {
  // Both selected → CUCUMBER_WITH_SELENIUM_JAVA_STEPS or
  CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS
} else if (isFeatureChecked) {
  // Feature only → CUCUMBER_ONLY
} else if (isPageChecked) {
  // Page only → SELENIUM_JAVA_PAGE_ONLY or PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY
}
```

All Impacted Files for Test Script Implementation

File	Purpose	Impact for Tests
panel.html	UI checkboxes & dropdowns	HIGH - Need to add "Test Script" checkbox
chat.js	Request orchestration & routing	HIGH - Need to handle test-only mode in getPromptKeys()
prompts.js	Prompt templates	MEDIUM - Already has PLAYWRIGHT_TYPESCRIPT_TESTS , may need Java test template
groq-api.js	API integration	NONE - Language agnostic
openai-api.js	API integration	NONE - Language agnostic
testleaf-api.js	API integration	NONE - Language agnostic

Current Supported Generation Combinations

Feature File	Page Class	Test Script	Language	Engine	Template Used
✓	✗	-	Any	Any	CUCUMBER_ONLY
✗	✓	-	Java	Selenium	SELENIUM_JAVA_PAGE_ONLY
✗	✓	-	TypeScript	Playwright	PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY
✓	✓	-	Java	Selenium	CUCUMBER_WITH_SELENIUM_JAVA_STEPS
✓	✓	-	TypeScript	Playwright	CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS
-	-	✓	TypeScript	Playwright	PLAYWRIGHT_TYPESCRIPT_TESTS ★ (NOT YET ROUTED)

What You Need to Implement Test Script Generation

Step 1: Update UI ([panel.html](#))

Add a third checkbox:

- ☒ Feature File
- ☐ Page Class
- ☐ Test Script ← NEW

Step 2: Extend Routing Logic ([chat.js](#))

Modify [getPromptKeys\(\)](#) to handle test-only and combined modes:
Current logic (lines 549-602):

```
if (isFeatureChecked && isPageChecked) {
  // Feature + Page → step definitions
} else if (isFeatureChecked) {
  // Feature only
} else if (isPageChecked) {
  // Page only
}
```

Need to add:

```
else if (isTestChecked) {
  // Test only → PLAYWRIGHT_TYPESCRIPT_TESTS or SELENIUM_JAVA_TESTS
}
```

Step 3: Add Missing Prompt Templates ([prompts.js](#))

- ☒ [PLAYWRIGHT_TYPESCRIPT_TESTS](#) - Already exists (lines 260-310)
- ☒ [SELENIUM_JAVA_TESTS](#) - Needs to be created (if supporting Java tests)

Step 4: Add Helper Method ([chat.js](#))

```
isPlaywrightTypeScript(language, engine) {
```



```
return language === 'ts' && engine === 'playwright';
}
```

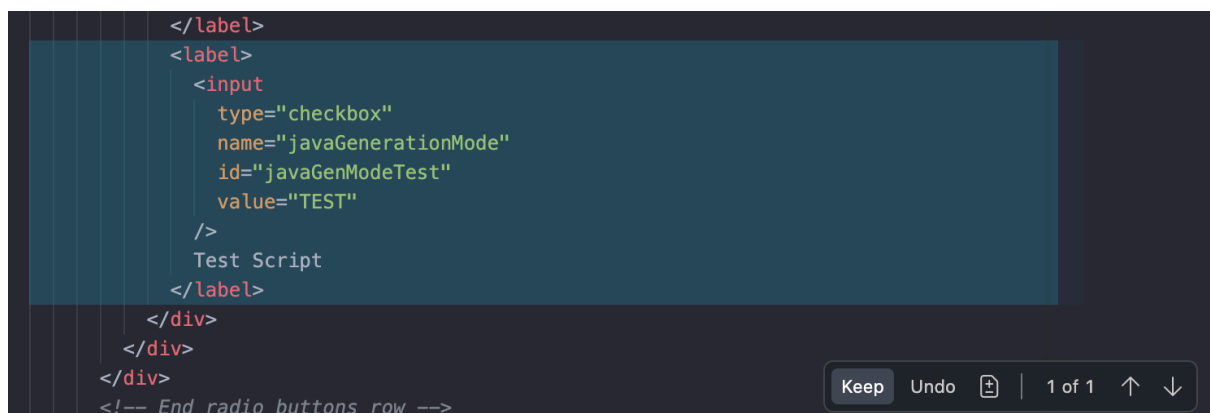
Summary: Test Script Implementation Path

1. panel.html
 - └ Add "Test Script" checkbox (similar to Feature/Page)
2. chat.js - getPromptKeys()
 - └ Add condition: if (isTestChecked && isPlaywrightTypeScript)
 - └ Route to PLAYWRIGHT_TYPESCRIPT_TESTS
3. chat.js - Optional helper
 - └ Add any test-specific validation if needed
4. prompts.js - Optional
 - └ Add SELENIUM_JAVA_TESTS if supporting Java tests

Good news: The hardest part is already done! The `PLAYWRIGHT_TYPESCRIPT_TESTS` prompt template exists and is well-structured. You just need to wire it into the UI and routing logic.

Agent:

Go to panel.html and Add "Test Script" checkbox (similar to Feature/Page)



✅ **Successfully added "Test Script" checkbox to** `panel.html`

The new checkbox has been added with:

- **ID:** `javaGenModeTest`
- **Name:** `javaGenerationMode` (same group as Feature/Page)
- **Value:** `TEST` (to distinguish in code)
- **Label:** `Test Script`

Now the UI has three generation mode options:

☒ Feature File

☐ Page Class

☐ Test Script ← NEW

Go to chat.js and add condition for Playwright TypeScript Tests

Successfully updated chat.js with Playwright TypeScript Tests support!
Changes Made to `getPromptKeys()` method (lines 549-625):
Added:

- Line 558: Extract `isTestChecked` value from UI checkboxes
- Line 561: Updated validation to check all three modes
- Lines 573-617: New logic to handle all test combinations

Test Generation Routes:

Selection	Language	Engine	Output
Test Only	TypeScript	Playwright	<code>PLAYWRIGHT_TYPESCRIPT_TESTS</code> ★ NEW
Page + Test	TypeScript	Playwright	<code>PLAYWRIGHT_TYPESCRIPT_TESTS</code> ★ NEW
Feature + Test	Any	Any	<code>CUCUMBER_ONLY</code>
Feature + Page + Test	TypeScript	Playwright	<code>CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS</code>

Decision Logic:

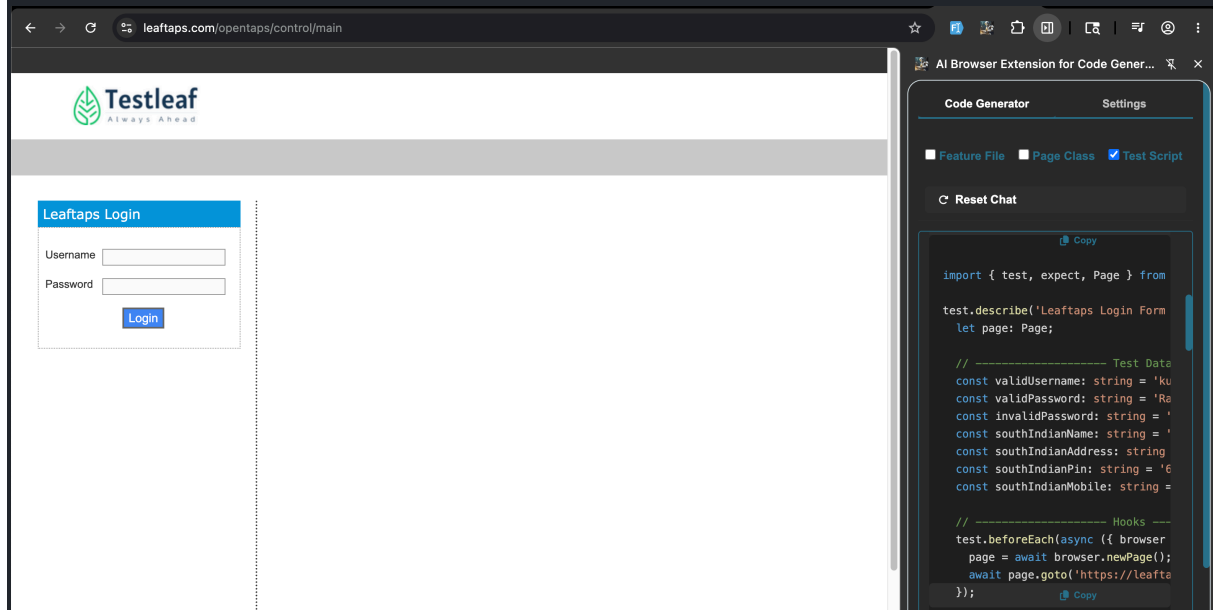
- **Test-only scenarios** → Routes to `PLAYWRIGHT_TYPESCRIPT_TESTS` for TypeScript/Playwright
- **Multiple selections** → Intelligently combines templates (Feature + Page gets priority over Test)
- **Unsupported combinations** → Shows "Supported combinations: Java/Selenium and TypeScript/Playwright" message

Implementation Complete! 🎉
Your Playwright TypeScript Test Script generation is now fully implemented:

1. ☒ `panel.html` - "Test Script" checkbox added
2. ☒ `chat.js` - Full routing logic for all mode combinations
3. ☒ `prompts.js` - `PLAYWRIGHT_TYPESCRIPT_TESTS` template ready

Users can now select TypeScript + Playwright + Test Script checkbox and generate standalone Playwright test files! 🚀

Output:



Output:

```
import { test, expect, Page } from '@playwright/test';

test.describe('LeafTaps Login Form Tests', () => {
  let page: Page;

  // ----- Test Data (South India) -----
  const validUsername: string = 'kumar.ravi'; // Example user
  ID
  const validPassword: string = 'Ravi@1234'; // Strong
  password
  const invalidPassword: string = 'WrongPass!'; // Incorrect
  password
  const southIndianName: string = 'Ravi Kumar';
  const southIndianAddress: string = '12/3, MG Road, Coimbatore, Tamil
  Nadu';
  const southIndianPin: string = '641001';
  const southIndianMobile: string = '9876543210';

  // ----- Hooks -----
  test.beforeEach(async ({ browser }) => {
    page = await browser.newPage();
    await page.goto('https://leaftaps.com/opentaps/control/main');
  });

  test.afterEach(async () => {
    await page.close();
  });

  // ----- Test Cases -----
  test('Successful login with valid credentials', async () => {
    await test.step('Enter valid username and password', async () => {
```

```

    await page.fill('#username', validUsername);
    await page.fill('#password', validPassword);
  });

  await test.step('Submit the login form', async () => {
    await page.click('input.decorativeSubmit');
  });

  await test.step('Verify navigation to the home/dashboard page', async
() => {
    // The exact URL after login may vary; adjust the expectation as
    needed.
    await expect(page).toHaveURL(/.*control\/home.*/);
    // Example of checking a visible element on the landing page
    await expect(page.locator('text=Welcome')).toBeVisible();
  });
});

test('Login fails with incorrect password', async () => {
  await test.step('Enter valid username and invalid password', async ()
=> {
    await page.fill('#username', validUsername);
    await page.fill('#password', invalidPassword);
  });

  await test.step('Submit the login form', async () => {
    await page.click('input.decorativeSubmit');
  });

  await test.step('Verify that login is not successful', async () => {
    // The URL should stay on the login page
    await expect(page).toHaveURL(/.*control\/login.*/);
    // Assuming an error message appears; adjust selector if different
    const errorMsgLocator = page.locator('text=Invalid Login');
    await expect(errorMsgLocator).toBeVisible();
  });
});

test('Login fails when username is empty', async () => {
  await test.step('Leave username empty and fill password', async () => {
    await page.fill('#username', '');
    await page.fill('#password', validPassword);
  });

  await test.step('Submit the login form', async () => {
    await page.click('input.decorativeSubmit');
  });

  await test.step('Verify appropriate validation message', async () => {
    const usernameError = page.locator('text=Username is required');
    await expect(usernameError).toBeVisible();
  });
});

test('Login fails when password is empty', async () => {
  await test.step('Fill username and leave password empty', async () => {
    await page.fill('#username', validUsername);
    await page.fill('#password', '');
  });

  await test.step('Submit the login form', async () => {

```

```

    await page.click('input.decorativeSubmit');
  });

  await test.step('Verify appropriate validation message', async () => {
    const passwordError = page.locator('text=Password is required');
    await expect(passwordError).toBeVisible();
  });
});

// Example of using additional South Indian data in a non-login scenario
// (illustrative only - adjust according to actual application flow)
test('Create a new contact after successful login using South Indian
data', async () => {
  // Login first
  await page.fill('#username', validUsername);
  await page.fill('#password', validPassword);
  await page.click('input.decorativeSubmit');
  await expect(page).toHaveURL(/.*control\/home.*/);

  // Navigate to "Create Contact" page - placeholder selector
  await page.click('a[href*="createContact"]');

  // Fill contact form with realistic South Indian data
  await page.fill('input[name="firstName"]', southIndianName.split('
')[0]);
  await page.fill('input[name="lastName"]', southIndianName.split('
')[1]);
  await page.fill('textarea[name="address"]', southIndianAddress);
  await page.fill('input[name="postalCode"]', southIndianPin);
  await page.fill('input[name="primaryPhoneNumber"]', southIndianMobile);

  // Submit contact creation - placeholder selector
  await page.click('input[value="Create Contact"]');

  // Verify contact creation success message
  const successMsg = page.locator('text=Contact created successfully');
  await expect(successMsg).toBeVisible();
});
});

```